Below is the complete README documentation for the Qloo_Project repository. This document outlines the project's purpose, main features, how to configure the project (including the special API key provided for the hackathon), how to install the dependencies, and how to run and use the project.

Qloo_Project

Introduction

Qloo_Project is a restaurant recommendation chatbot API designed during a hackathon for Devpost. The project integrates with the Qloo hackathon API endpoint (https://hackathon.api.qloo.com) and leverages generative AI capabilities via Google's Gemini generative model (through the langchain and google.generativeai libraries) to process user messages and extract relevant keywords for restaurant preferences. Built with a Python backend using FastAPI and a frontend powered by React with Vite, this application demonstrates how to combine real-time data extraction, recommendation retrieval, and conversational AI for a seamless user experience.

The project relies on robust integration with Qloo's recommendation and insights services as well as Firebase for storing user sessions and feedback. During the hackathon, a dedicated API key (qloo_api) was provided to test and utilize the Qloo endpoints. Key endpoints in the backend allow for restaurant details retrieval, chat session history, and a health check service.

Features

Key Features:

- **Conversational ChatBot API:**
  • Processes user input using generative AI to decide whether to recommend restaurants based on past visits or for new dining experiences. filecite turn0file1

- **Restaurant Recommendations:**
  • Integrates with Qloo to get recommendations and insights based on location and food preferences. The service dynamically searches using multiple radius values and tag filtering for accurate recommendations. filecite turn0file2

- **Firebase Integration:**
  • Stores and retrieves chat sessions, user profiles, and feedback using Firebase's Firestore service for persistent state management. filecite turn0file0

- **Generative AI Assistance:**
  • Uses Google Generative AI (Gemini) to extract keywords from user prompts and determine recommendation type (old visits vs. new search). filecite turn0file1, filecite turn0file7

- **Frontend Application:**
  • A modern React-based UI (using Vite) allows users to interact with the chat interface and view recommendations.

- **Testing Endpoints:**
  • Contains endpoints such as `/api/test-qloo` for quickly validating the connection with the Qloo API (with fallback solutions in case the API key isn't available). filecite turn0file10

Configuration

Environment Variables and API Keys:

Before running the project, ensure you create a `.env` file in the project root or configure your environment with the following variables:

- **QLOO_API_KEY:**
  • Your dedicated API key for the hackathon provided by Qloo.
  Example:
  `QLOO_API_KEY=your_qloo_api_key_here`

- **FIREBASE_API_KEY, FIREBASE_AUTH_DOMAIN, FIREBASE_PROJECT_ID, FIREBASE_STORAGE_BUCKET, FIREBASE_MESSAGING_SENDER_ID, FIREBASE_APP_ID:**
  • Firebase configuration credentials required for user session and feedback storage. These are imported into the backend and used to initialize the Firebase Admin SDK.
  (See configuration in the backend where `firebase_config` is defined using environment variables.) filecite turn0file0

- **GEMINI_API_KEY:**
  • API key needed for the Gemini generative model which powers language understanding and responses.
  Example:
  `GEMINI_API_KEY=your_gemini_api_key_here`

Other configurations include:

- **Endpoints:**
  • The Qloo API endpoints such as `/v2/insights` and `/v2/tags` are configured in the code.
  • The base URL for Qloo is hard-coded as `https://hackathon.api.qloo.com`.

Make sure that these keys are set up correctly in your environment so that the backend can access Qloo's services and Firebase without issues.

Installation

The project consists of a Python-based backend and a React (TypeScript) frontend. Follow these steps to set up the environment:

Backend Setup (Python)

1. ## Clone the repository: Code:

   ## git clone https://github.com/RehanAnsari17/Qloo_Project.git cd Qloo_Project/backend

2. ## Create and activate a virtual environment: Code:

   ## python -m venv venv source venv/bin/activate # On Windows use: venv\Scripts\activate

3. ## Install Python dependencies: Code:

   ## pip install -r requirements.txt

   • The repository uses FastAPI, requests, httpx, google.generativeai, python-dotenv, and Firebase Admin SDK among others.
   • Some additional helper packages (like Naked CLI framework) are also installed in the environment.

4. **Configure Environment Variables:** • Place your environment variables into a `.env` file at the backend root as described in the Configuration section.

5. ## Run the Backend Server: Code:

   ## uvicorn main:app --reload

   • This command will start your FastAPI backend on port 8000 by default.

Frontend Setup (React with Vite & TypeScript)

1. ## Navigate to the frontend directory: Code:

cd ../frontend

2. ## Install Node.js dependencies: Code:

npm install

• The frontend package includes settings for React, TypeScript, and ESLint (see eslint.config.js for linting rules). filecite turn0file10

3. ## Run the Frontend Development Server: Code:

npm run dev

• This will launch the Vite development server, accessible at the URL shown in the terminal (usually http://localhost:3000).

Usage

After successful installation, you can interact with the project as follows:

Frontend Interaction

- Visit the frontend URL in your web browser.
- In the chat interface, type your restaurant preferences or queries.
- The interface will call the backend API endpoints to fetch restaurant recommendations based on either: • Prior visits (if the conversation implies previous restaurant ratings/feedback)
  • A fresh recommendation based on dynamically extracted keywords and location.
- React components handle displaying the restaurant image, rating, address, and other details.

Backend API Endpoints

The backend server exposes several endpoints for different functionalities:

- **GET /api/restaurant-details/{restaurant_id}:**
  • Retrieves detailed information about a specific restaurant.
  • Uses insights from Qloo based on entity IDs to fetch full details. filecite turn0file4

- **GET /api/chat-history:**
  • Returns a list of previous chat sessions including session ID, user name, creation date, and a preview of the session messages.

- **GET /api/chat-session/{session_id}:**
  • Returns the detailed conversation history for a given session ID.

- **GET /api/health:**
  • Health check endpoint that returns the status and the current timestamp.

- **GET /api/test-qloo:**
  • Dedicated testing endpoint to validate connectivity to the Qloo API. If successful, returns sample insights data obtained from Qloo. filecite turn0file10

Additional Details

- **Dynamic Tag Filtering:**
  The backend also contains logic to process keywords extracted from user input, convert them into tag URNs, and use these for fetching restaurant insights. Fallback mechanisms are in place to retry searches if initial results are not available. This ensures that users always receive recommendations even if the first query returns empty. filecite turn0file7

- **Firebase Integration for Feedback:**
  Chat sessions and user feedback are stored in a Firebase Firestore, and there is a script (gather.py) that demonstrates how feedback can be aggregated and displayed for debugging purposes.

By ensuring proper configuration of all API keys and environment variables, this project can be used as a full-stack demonstration of integrating a cutting-edge recommendation service with conversational AI. Enjoy exploring Qloo_Project and extend its functionalities as desired!

This README covers the complete setup, configuration, and usage details based solely on the content from the repository's source files filecite turn0file0, filecite turn0file2, and others.